

Design Pattern

1. Singleton - гарантира, че от даден клас съществува само един обект и предоставя глобален достъп до него.

Пример: Искаме програмата да има само един Logger:

```
class Logger:
```

```
{ private:
```

```
    Logger() = default;
```

→ забраняваме създаването на обекти, защото конструкторът е private, тоест Logger logger; → дава грешка

```
public:
```

```
    Logger(const Logger& other) = delete;
```

```
    Logger& operator=(const Logger& other) = delete;
```

копирането и инициализирането е забранено

```
    Logger(Logger&& other) noexcept = delete;
```

```
    Logger& operator=(Logger&& other) noexcept = delete;
```

```
    static Logger& getInstance()
```

```
    { static Logger instance;
```

```
      return instance;
```

```
    }
```

```
    void log(const string message)
```

```
    { std::cout << message << "\n"; }
```

```
};
```

Достъпваме така:

```
Logger::getInstance().log(...);
```

създава локална статична променлива само веднъж

2. Iterator - позволява да обхождаме елементите на колекция, без да знаем как са съхранени вътрешно.

Пример: Итератор за темплейтен масив:

```
template <typename T>
class DynamicArray {
    T* data;
    int size;
    int capacity;
public:
    class Iterator → вложен клас
    {
        T* ptr; → указател към текущия елемент
    public:
        explicit Iterator(T* p): ptr(p) {}
        T& operator*() const { return *ptr; } → достъп до стойността
        T* operator->() const { return ptr; } → адресен оператор
        Iterator& operator++() {
            ++ptr; → prefix
            return *this;
        }
        Iterator operator++(int) → postfix
        {
            Iterator temp = *this;
            ++ptr;
            return temp;
        }
        // Сравнения
    };
    Iterator begin() { return Iterator(data); }
    Iterator end() { return Iterator(data+size); }
```

→ begin и end
вече в класа
Dynamic Array

18.11
Factory - създава обекти вместо клиентския код да използва директно конкретните конструктори.
- Клиентът казва какво иска, а Factory решава кой конкретен клас да създаде.

Пример:

```
class Animal {  
public:  
    virtual void makeSound() const = 0;  
    virtual ~Animal() = default;  
};  
class Dog: public Animal  
{ public:  
    void makeSound() override { cout << "woof"; }  
};  
class Cat: public Animal  
{ public:  
    void makeSound() override { cout << "meow"; }  
};  
class AnimalFactory {  
public:  
    static unique_ptr<Animal> create(const string& type)  
    { if (type == "dog") { return make_unique<Dog>(); }  
      if (type == "cat") { return make_unique<Cat>(); }  
      throw ...;  
    }  
};
```

→ базов клас

} наследници

Използваме при:

- базов клас и много наследници
- искаме създаване на едно място
- искаме да скрием конкретните класове
- конкретния тип се избира от входа

4. Observer - когато един обект промени състоянието си, той известява всички абонирани наблюдатели.

Пример: Когато Poster създаде публикация, всички Reader обекти получават известие.

class Subscriber { ← Observer (наблюдател) интерфейс
public:

virtual void update(const string& data) = 0;

virtual ~Subscriber() = default;

};

class Reader : public Subscriber { ← Конкретен наблюдател
private:

string name;

public:

explicit Reader(const string& name): name(name) {}

void update(const string& data) override { cout << name << "received" << data; }

};

class Publisher {

private:

vector<Subscriber> subscribers;

public:

void subscribe(Subscriber* subscriber)

{ // проверка за subscriber

subscribers.push_back(subscriber);

}

void unsubscribe(Subscriber* subscr)

{ auto newEnd = std::remove(subscribers.begin(), subscribers.end(), subscr);

subscribers.erase(newEnd, subscribers.end());

}

void notify(const string& data) {

for(Subscriber* sub : subscribers)

sub->update(data);

};

не притежава, само наблюдава
пази указател към статичните

5.
5. Prototype - създава нов обект като копие на съществуващ обект
- при полиморфни йерархии.

```
class Animal {
```

```
public:
```

```
virtual unique_ptr<Animal> clone() const = 0; → решение чрез clone()  
virtual ~Animal() = default;
```

```
};
```

```
class Dog: public Animal {
```

```
private:
```

```
    string name;
```

```
public:
```

```
explicit Dog(const string& name): name(name) {}
```

```
unique_ptr<Animal> clone() const override
```

```
{ return make_unique<Dog>(*this); }
```

```
};
```

← Наследник

6. Visitor - позволява добавяне на нови операции върху класова йерархия, без да променяме самите класове от йерархията

Пример:

```
class Bank;  
class Company;  
class Restaurant;
```

} декларираме ги, се
ще ги използваме

```
class IVisitor {
```

→ IVisitor интерфейс

```
public:
```

```
virtual void visitBank(const Bank* bank) = 0;  
virtual void visitCompany(const Company* company) = 0;  
virtual void visitRestaurant(const Restaurant* r) = 0;  
virtual ~IVisitor() = default;  
};
```

```
class IClient {
```

→ базовият клас

```
public:
```

```
virtual string getName() = 0;  
virtual void accept(IVisitor* visitor) = 0;  
virtual ~IClient() = default;  
};
```

```
class Bank : public IClient {
```

→ Всеки наследник се
представя
чрез
accept и
след това се решава кое
visit да се изпълни

```
public:
```

```
string getName() const override { return "Bank"; }  
void accept(IVisitor* visitor) override  
{ visitor->visitBank(this); }  
};
```

```
class Company : public IClient {
```

```
void accept(-//-) override  
{ visitor->visitCompany(this); }  
};
```

Използваме когато:

- често добавяме нови операции
- операцията са различни за конкретни типове

Конкретни visitor-и:

Ще наследяват IVisitor и ще предефинират методите visitBank, visitCompany и т.н.

1. Command - превръща действие в отделен обект

```
class Command {
public:
    virtual void execute()=0;
    virtual void undo()=0;      → може и redo()=0;
    virtual ~Command()=default;
};
```

Конкретни команди:

Наследяват Command и предефинират методите

```
class Editor {
    vector<unique_ptr<Command>> history;
public:
    void execute(unique_ptr<Command> command)
    {
        command->execute();
        history.push_back(std::move(command));
    }
    void undo() {
        if (history.empty()) { return; }
        history.back()->undo();
        history.pop_back();
    }
};
```

Използваме когато:

- искаме undo()
- искаме история на действията
- искаме опашка от операции

8. Memento - запазва моментно състояние на обект, така се по-късно да може да бъде възстановено.

Пример:

```
class ICommand {  
public:  
    virtual void execute()=0;  
    virtual ~ICommand()=default;  
};
```

→ Интерфейс само с execute()

```
class Document {  
    string text;  
public:
```

→ Originator - обектът, чето състояние пази

// негови си методи → write(string & word)...

```
std::unique_ptr<Document> clone() const  
{ return make_unique<Document>(*this); }  
};
```

```
class WriteCommand: public ICommand  
{ Document & doc;  
    string word;  
public:  
    // конструктор  
    void execute() override { doc.write(word); }  
};
```

→ Конкретна команда само execute()

```
class TextEditor {  
    std::unique_ptr<Document> doc;  
    stack<unique_ptr<Document>> undoStack; → преди execute  
    stack<unique_ptr<Document>> redoStack; → преди undo  
public:  
    execute(cmd) → 1. doc → clone & undoStack, 2. изчиства redoStack, 3. cmd → execute  
    undo... → 1. redoStack.push(doc → clone()); 2. doc = move(undoStack.top());  
              3. undoStack.pop()  
    redo... → 1. undoStack.push(—) 2. doc = move(redoStack.top());  
              3. redoStack.pop()
```